

Arpan Adhikari

231309

TOL UT Question Ans.

Q1a) soln

Given language, $L = \{w \in \{0,1\}^* : w \text{ contains '0110' or '1001' as substring}\}$

def, the NFA be $(Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$

$\Sigma = \{0,1\}$

$q_0 = q_0$

$F = \{q_4\}$

$\delta =$

$Q \backslash \Sigma$	0	1
q_0	q_1	q_5
q_1	q_2	q_4
q_2	q_3	q_4
q_3	q_4	q_4
q_4	q_4	q_4
q_5	q_6	q_4
q_6	q_7	q_4
q_7	q_4	q_4

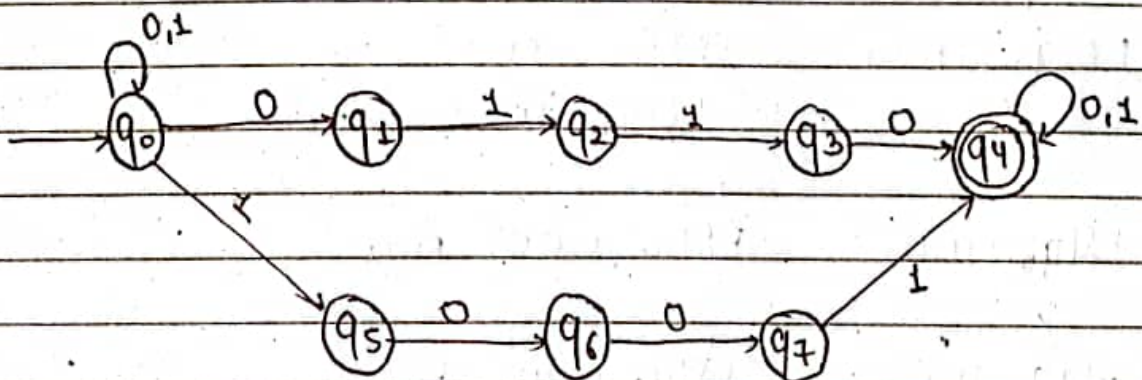


Fig. NFA State diagram

$\delta:$	Q/Σ	0	1
	q_0	$\{q_0, q_1\}$	$\{q_0, q_5\}$
	q_1	-	q_2
	q_2	-	q_3
	q_3	q_4	-
	q_4	q_4	q_4
	q_5	q_6	-
	q_6	q_7	-
	q_7	-	q_4

Testing:

For 01011100,

$\delta(q_0, 01011100)$

$\vdash \delta(q_0, 1011100)$

$\vdash \delta(q_1, 1011100)$

$\vdash \delta(q_2, 011100)$ Reject

$\vdash \delta(q_0, 011100)$

$\vdash \delta(q_5, 011100)$

$\vdash \delta(q_6, 11100)$ Reject

$\vdash \delta(q_0, 11100)$

$\vdash \delta(q_1, 11100)$

$\vdash \delta(q_2, 1100)$ Reject

$\vdash \delta(q_0, 1100)$

$\vdash \delta(q_5, 1100)$ Reject

$\vdash \delta(q_0, 100)$

$\vdash \delta(q_5, 1100)$ Reject

$\vdash \delta(q_0, 00)$

$\vdash \delta(q_5, 00)$

$\vdash \delta(q_6, 00)$

$\vdash \delta(q_0, 0)$

$\vdash \delta(q_1, 0)$

$\vdash \delta(q_7, 0)$ Reject

$\vdash \delta(q_0, \epsilon)$

$\vdash \delta(q_1, \epsilon)$

Reject

Reject

Reject

Hence, the string is rejected.

For 1010011,

$\delta(q_0, 1010011)$

$\vdash \delta(q_0, 010011)$

$\vdash \delta(q_0, 10011)$

$\vdash \delta(q_5, 0011)$

$\vdash \delta(q_6, 011)$

$\vdash \delta(q_7, 11)$

$\vdash \delta(q_4, 1)$

$\vdash \delta(q_4, \epsilon)$

Accept Ans

Q1b)

Conversion of NFA to DFA:

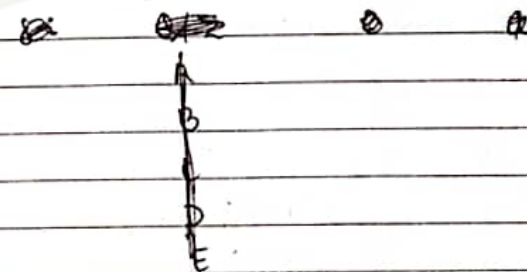
Given NFA is $(Q, \Sigma, \delta, q_0, F)$

$Q = \{A, B, C, D, E\}$

$\Sigma = \{0, 1\}$

$q_0 = A$

$F = \{C, E\}$



Finding E-closure,

$E(A) = \{A, B, D\}$

$E(B) = \{B, D\}$

$E(C) = \{C, E\}$

Let start state of equivalent DFA is E-closure of start state of NFA
i.e. $\{A, B, D\}$

For $\{A, B, D\}$,

$$\begin{aligned} & \delta'(\{A, B, D\}, 0) \\ &= E(D) \cup E(C) \cup E(E) \\ &= \{C, D, E\} \text{ N.S.} \end{aligned}$$

$$\begin{aligned} & \delta'(\{A, B, D\}, 1) \\ &= E(A) \cup E(D) \cup E(C) \cup \emptyset \\ &= \{A, B, D\} \cup \{D\} \cup \{C, E\} \cup \emptyset \\ &= \{A, B, C, D, E\} \text{ N.S.} \end{aligned}$$

For $\{C, D, E\}$,

$$\begin{aligned} & \delta'(\{C, D, E\}, 0) \\ &= \emptyset \cup E(E) \cup E(C) \\ &= \{C, E\} \text{ N.S.} \end{aligned}$$

$$\begin{aligned} & \delta'(\{C, D, E\}, 1) \\ &= \emptyset \cup \emptyset \cup E(C) \\ &= \{C, E\} \end{aligned}$$

For $\{A, B, C, D, E\}$,

$$\begin{aligned} & \delta'(\{A, B, C, D, E\}, 0) \\ &= E(D) \cup E(C) \cup \emptyset \cup E(E) \cup E(C) \\ &= \{D\} \cup \{C, E\} \cup \{E\} \cup \{C, E\} \\ &= \{C, D, E\} \end{aligned}$$

$$\begin{aligned} & \delta'(\{A, B, C, D, E\}, 1) \\ &= E(A) \cup E(D) \cup E(C) \cup \emptyset \cup \emptyset \cup E(C) \\ &= \{A, B, D\} \cup \{D\} \cup \{C, E\} \cup \{C, E\} \\ &= \{A, B, C, D, E\} \end{aligned}$$

For $\{C, E\}$,

$$\begin{aligned} & \delta'(\{C, E\}, 0) \\ &= \emptyset \cup E(C) \\ &= \{C, E\} \end{aligned}$$

$$\begin{aligned} & \delta'(\{C, E\}, 1) \\ &= \emptyset \cup E(C) \\ &= \{C, E\} \end{aligned}$$

No more new states. so, process terminates.

Now, equivalent DFA be $D = (Q', \Sigma', \delta', q_0', F')$
where,

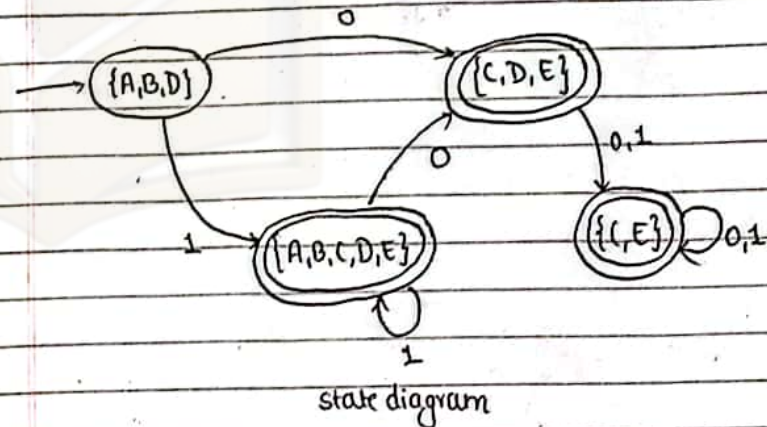
$$Q' = \{\{A, B, D\}, \{C, D, E\}, \{A, B, C, D, E\}, \{C, E\}\}$$

$$\Sigma' = \{0, 1\}$$

$$q_0' = \{A, B, D\}$$

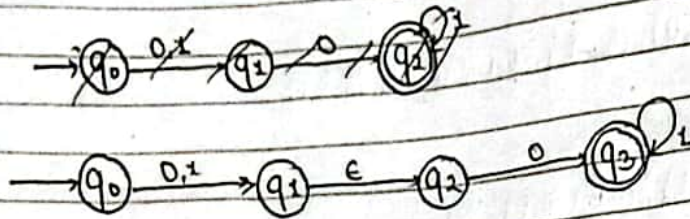
$$F' = \{\{C, D, E\}, \{A, B, C, D, E\}, \{C, E\}\}$$

δ' : Q'/Σ'	0	1
$\{A, B, D\}$	$\{C, D, E\}$	$\{A, B, C, D, E\}$
$\{C, D, E\}$	$\{C, E\}$	$\{C, E\}$
$\{A, B, C, D, E\}$	$\{C, D, E\}$	$\{A, B, C, D, E\}$
$\{C, E\}$	$\{C, E\}$	$\{C, E\}$

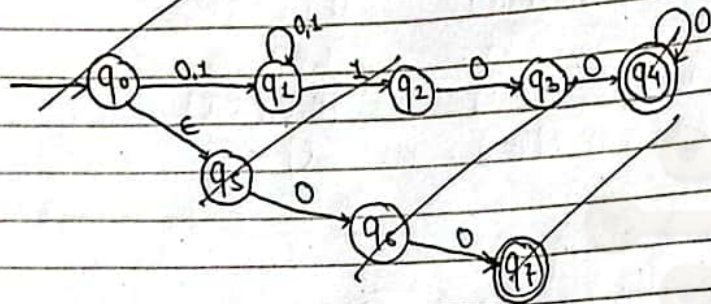


Q2a) Convert RE to E-NFA.

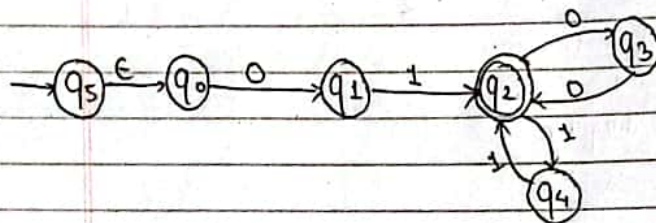
(i) $10^+10^+1^*$



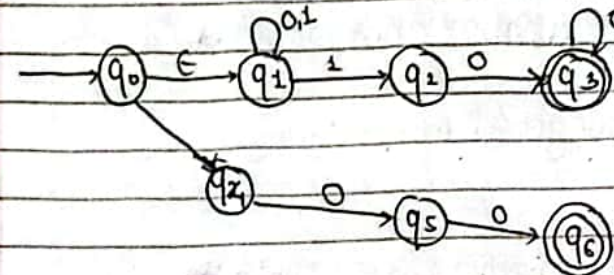
(ii) $00^+(0+1)^*100^*$



(iii) $01(00^+11)^*$



(ii) $00^+(0+1)^*100^*$



2b) Here,

$$\begin{aligned} A &= \epsilon & \text{--- (i)} \\ B &= A.a + D.a & \text{--- (ii)} \\ C &= B.b + D.b & \text{--- (iii)} \\ D &= A.b + B.a + C.a + C.b & \text{--- (iv)} \end{aligned}$$

Putting values of B & C in (iv),

$$D = A.b + A.a.a + D.a.a + B.b.a + D.b.a + B.b.b + D.b.b$$

$$\text{or, } D = b + aa + B.ba + B.bb + D.aa + D.ba + D.bb \quad [\text{using (i)}]$$

$$\text{or, } D = b + aa + A.aba + D.aba + A.abb + D.abb + D.aa + D.ba + D.bb \quad [\text{using (ii)}]$$

$$\text{or, } \underbrace{D}_{R} = \underbrace{(b + aa + aba + abb)}_Q + \underbrace{D}_{R} \underbrace{(aba + abb + aa + ba + bb)}_P$$

Using Ardens theorem, Comparing with $R = Q + RP$,

$$R = D$$

$$Q = (b + aa + aba + abb)$$

$$P = (aba + abb + aa + ba + bb)$$

We get, $R = QP^*$

$$D = (b + aa + aba + abb)(aba + abb + aa + ba + bb)^* \quad \text{--- (v)}$$

For (ii), Using (i) & (v) in (ii), we get,

$$B = a + (btaa + abatabb)(aba + abbtaa + ba + bb)^*a \quad \text{--- (vi)}$$

Hence, From (v) & (vi), we get (iii) as,
 $C = B.b + D.b$

$$RE = (a + (btaa + abatabb)(aba + abbtaa + ba + bb)^*a)b + (btaa + abatabb)(aba + abbtaa + ba + bb)^*ab$$

2c) no

closure properties of regular language:

- Regular language exhibit structural properties known as closure properties, meaning that applying specific operations to regular language produces another regular language.
- The class of language occupied by finite automata is closed under:

(i) Union

The union of two regular languages L_1 and L_2 denoted by $L_1 \cup L_2$ is the set of strings belonging to L_1, L_2 or both.

(ii) concatenation

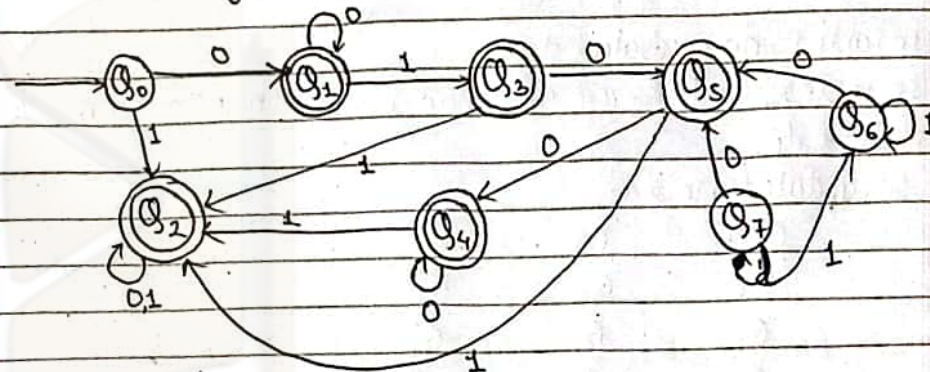
The concatenation of L_1 and L_2 denoted by $L_1.L_2$ is the set of strings formed by appending any string from L_2 to any string from L_1 .

(iii) Kleene closure

The Kleene closure of a language L , denoted L^* , is the set of all strings formed by concatenating zero or more strings from L (including the empty string ϵ).

Q3a) no

The initial diagram is as,



state diagram

Here, Q_7 and Q_6 are unreachable states. So, remove them, constructing transition table of remaining states,

Q/\Sigma	0	1
Q_0	Q_1	Q_2
Q_1	Q_1	Q_3
Q_2	Q_2	Q_2
Q_3	Q_5	Q_2
Q_4	Q_4	Q_2
Q_5	Q_4	Q_2

Dividing above table into two table as follows:

Q/Σ	0	1
Q_0	Q_1	Q_2

Table-1

Q/Σ	0	1
Q_1	Q_1	Q_3
Q_2	Q_2	Q_1
Q_3	Q_5	Q_2
Q_4	Q_4	Q_2
Q_5	Q_4	Q_2

Table-2

- In Table 1, no equivalent states.
- In Table 2, Q_1 & Q_5 are equivalent because they have same o/p to same i/p.

So, update table 2 as,

Q/Σ	0	1
Q_1	Q_1	Q_3
Q_2	Q_2	Q_2
Q_3	Q_4	Q_2
Q_4	Q_4	Q_2

Table-3

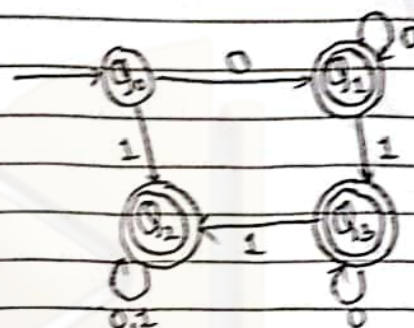
- In Table-3, Q_3 & Q_4 are equivalent. So, updated table is,

Q/Σ	0	1
Q_1	Q_1	Q_3
Q_2	Q_2	Q_2
Q_3	Q_3	Q_2

Hence, Final table is (after combining):

Q/Σ	0	1
Q_0	Q_1	Q_2
Q_1	Q_1	Q_3
Q_2	Q_2	Q_2
Q_3	Q_3	Q_2

Minimized diagram is,



(Q.3b)

Finite Automata is the model of computation that accepts/rejects language.

Given language is $L = \{w \in \{a,b\}^* : w \text{ contains 'bb' or 'bab' as substring}\}$

Let DFA be $D = (Q, \Sigma, \delta, q_0, F)$

where, $Q = \{q_0, q_1, q_2, q_3\}$

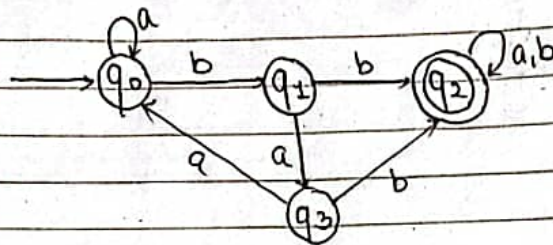
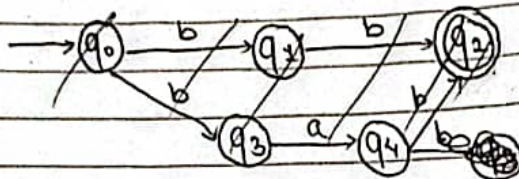
$\Sigma = \{a, b\}$

$q_0 = q_0$

$F = \{q_2\}$

S: Q/Σ

	a	b
$\rightarrow q_0$	q_0	q_1
q_1	q_3	q_2
$*q_2$	q_2	q_2
q_3	q_1	q_2



Test:

(i) abaabb

$\delta(q_0, abaabb)$

$\vdash \delta(q_0, baabb)$

$\vdash \delta(q_1, aabb)$

$\vdash \delta(q_3, abb)$

$\vdash \delta(q_0, bb)$

$\vdash \delta(q_1, b)$

$\vdash \delta(q_2, \epsilon)$

Accept

(ii) baaba

$\delta(q_0, baaba)$

$\vdash \delta(q_1, aaba)$

$\vdash \delta(q_3, aba)$

$\vdash \delta(q_0, ba)$

$\vdash \delta(q_1, a)$

$\vdash \delta(q_3, \epsilon)$

Reject

Q40) Is NFA more powerful than DFA?

Ans

NFAs and DFAs are equally powerful in terms of computational capability as both recognize exactly the same class of languages (regular languages). This equivalence is proven by the fact that every NFA can be converted into an equivalent DFA, though the DFA may require exponentially more states.

1. Formal Equivalence Proof

Any NFA $M = (Q, \Sigma, \delta, q_0, F)$ can be converted to DFA $M' = (Q', \Sigma, \delta', q_0', F')$ where,

$Q' = 2^{Q_N}$ (all subsets of NFA states)

2. NFA Features Do not add power

While NFAs follow:

- ϵ -transitions

- Multiple transitions for a single input.

These features don't expand the set of recognizable languages beyond what DFAs can handle.

Examples illustrating equivalence:

Example 1: NFA:

Consider an NFA accepting strings ending in "01";

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

$F = \{q_2\}$

S: Q/Σ

	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

Equivalent DFA:

We obtain the DFA for above NFA as, $D = (Q', \Sigma, \delta', q_0', F')$

$$Q' = \{ \{q_0\}, \{q_0, q_1\}, \{q_0, q_2\} \}$$

$$\Sigma = \{0, 1\}$$

$$q_0' = q_0$$

$$F' = \{ \{q_0, q_2\} \}$$

δ' :

Q'/Σ	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_2\}$	$\{q_0\}$

This DFA accepts the same language as original NFA.

Q4b)

Soln

Given, $G = (V, T, P, S)$

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$S = \{S\}$$

$$P = \{ S \rightarrow AB$$

$$A \rightarrow aA$$

$$A \rightarrow a$$

$$B \rightarrow bB$$

$$B \rightarrow b \}$$

Using Left Most Derivation,

$$S \Rightarrow AB$$

$$\Rightarrow aAB$$

$$\Rightarrow aaAB$$

$$\Rightarrow aaaAB$$

$$\Rightarrow aaaaB$$

$$\Rightarrow aaaaBB$$

$$\Rightarrow aaaaBBB$$

$$\Rightarrow aaaaBBB$$

Using Right Most Derivation,

$$S \Rightarrow AB$$

$$\Rightarrow AbB$$

$$\Rightarrow AbbB$$

$$\Rightarrow Abbb$$

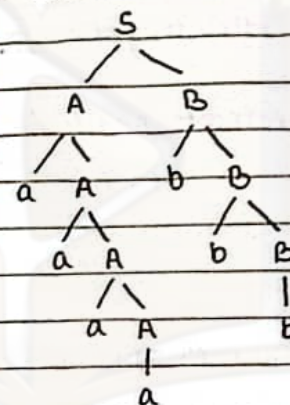
$$\Rightarrow aAbbb$$

$$\Rightarrow aaAbbb$$

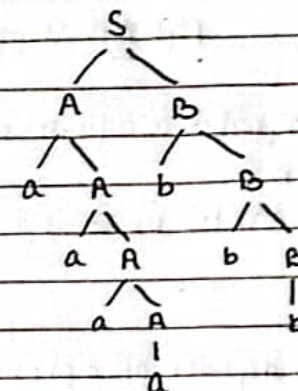
$$\Rightarrow aaaAbbb$$

$$\Rightarrow aaaaAbbb$$

Parse tree:



Parse tree:



Q5b)

Pumping Lemma for regular language

- Pumping Lemma says that all sufficient long words, in a regular language may be pumped i.e. have a middle section of word repeated an arbitrary no. of times to produce new word also lies within same language.

statement of pumping lemma:

- Let L be a regular language.
- w be any string, $w \in L$ with $|w| \geq n$; where n be integer constant.
- There are strings x, y, z such that
 $w = xyz$ where $|xy| \leq n$
 $|y| > 0$

then, $xy^iz \in L$ for all $i \geq 0$

Here, substring y is pumped.

5a)

Alphabet, String, Kleene Closure, Positive Closure & Language

- An alphabet is a finite, non-empty set whose elements are called symbols.
- It is denoted by Σ . e.g. $\Sigma = \{0, 1\}$
 $\Sigma = \{a, b\}$
- A string over an alphabet Σ is a finite sequence of symbols where each symbol is an element of Σ .
- It is denoted by w .
- The set of all strings over an alphabet Σ is called Kleene closure of Σ .
- The set of all strings over an alphabet Σ except empty string is called positive closure of Σ .
- Any set of strings over an alphabet Σ given some conditions is called language.